CAMPUSXCHANGE

Campus Marketplace for Students

TECHNICAL PROJECT REPORT

System Design & Object-Oriented Programming

Project Title	CampusXchange — Campus Marketplace for Students
Course	System Design & Object-Oriented Programming
Tech Stack	Next.js | Node.js | Express.js | MongoDB | Socket.IO | Razorpay
Repository	github.com/BineetKeshari2005/campusXchange
Live Demo	https://campus-xchange-sd.vercel.app/auth/login
Team Member	Bineet Keshari
Submission Date	27 April 2026

Submitted in partial fulfilment of course requirements

Table of Contents

1. Executive Summary

CampusXchange is a full-stack, real-time web platform purpose-built for university students to buy, sell, and exchange academic materials within a trusted campus community. The platform addresses the absence of a secure, student-exclusive marketplace by combining a modern Next.js frontend, a Node.js/Express.js backend, MongoDB for persistence, Socket.IO for real-time communication, and Razorpay for payment processing.

This report documents the complete technical design and implementation of CampusXchange, with emphasis on the application of Object-Oriented Programming principles, Design Patterns, SOLID principles, and System Design concepts as required by the course curriculum. The project was refactored from a basic MVC structure into a production-grade, five-layer architecture. All major deliverables — including the working application, GitHub repository, UML diagrams, and this report — have been completed.

2. Problem Statement & Objectives

2.1 Problem Statement

University students routinely need affordable textbooks, study notes, second-hand electronics, and academic accessories. Generic platforms such as OLX and Facebook Marketplace allow participation from external users, creating safety concerns, lack of trust, and an absence of campus-specific features. Students are left with no dedicated, secure channel to transact within their community.

Core pain points identified:

- No campus-exclusive marketplace — outsiders can contact students.
- No integrated payment system — transactions are cash-only and untracked.
- No real-time communication — buyers and sellers cannot negotiate instantly.
- No centralised listing management — sellers have no dashboard to track sales.

2.2 Project Objectives

1. Build a campus-exclusive authentication system using JWT so only registered students participate.
2. Implement a structured listing system with full-text search, category filters, price range, condition filter, sort, and pagination.
3. Provide real-time buyer-seller chat via Socket.IO within a secure, room-based architecture.
4. Integrate Razorpay payment gateway with HMAC signature verification and automated seller payout transfer.

5. Apply OOP, SOLID, Design Patterns, and System Design principles throughout the codebase.

3. Technology Stack

The following technologies were selected based on their industry adoption, suitability for real-time applications, and team familiarity.

Layer	Technology	Version / Notes	Purpose
Frontend	Next.js	v14 (App Router)	Server-side rendering, routing, UI
Styling	Tailwind CSS	v3	Utility-first responsive design
Animation	Framer Motion	v10	Page & component animations
Backend	Node.js	v22	JavaScript runtime
API Framework	Express.js	v4	REST API server
Real-time	Socket.IO	v4	Bidirectional live chat
Database	MongoDB	Atlas (cloud)	NoSQL document store
ODM	Mongoose	v8	Schema modelling for MongoDB
Auth	JWT	jsonwebtoken v9	Stateless authentication tokens
Payments	Razorpay	v2 API	Payment gateway + seller transfers
File Upload	Cloudinary	multer-storage-cloudinary	Image storage & CDN
Deployment	Vercel	Frontend	CI/CD + edge network
Deployment	Render	Backend	Node.js hosting

4. System Architecture

4.1 Layered Architecture (System Design)

The backend follows a strict five-layer architecture. Each layer communicates only with the layer directly below it. This separation of concerns is the foundation of all SOLID principles applied in this project.

Layer	Class / File	Responsibility	Talks To
Route	routes/index.js	Maps HTTP path to a controller function. Zero logic.	Controller
Controller	controllers/index.js	Reads req, calls service, sends res. Zero business logic.	Service
Service	services/*.js	All business logic: validation, hashing, token generation.	Repository
Repository	repositories/*.js	All MongoDB access. Services never call Mongoose directly.	Model
Model	models/*.js	Mongoose schema + instance methods (e.g. getPublicProfile).	MongoDB

4.2 Frontend Architecture

The Next.js frontend is organised into three layers in addition to the standard page/component structure:

Layer	Location	Purpose
Service Layer	src/services/api.js	All fetch() calls live here. Components never call fetch() directly. 7 service classes: AuthService, ListingService, ProfileService, SavedService, PaymentService, OrderService, ChatService.
Custom Hooks	src/hooks/index.js	useListings, useSaved, useProfile, useAuth, useOrders. Manage loading/error state. Components import hooks, not services directly.
Pages	src/app/**/page.jsx	Pure UI components. Receive data from hooks. No direct API calls anywhere in page components.

4.3 Real-Time Communication Architecture

Socket.IO is used for live buyer-seller chat. Key design decisions:

- JWT token verified on socket handshake — unauthenticated sockets are rejected immediately.

- Each conversation has its own room (conversationId). Users join only rooms they are party to.
- Unread message counts tracked per participant in the Conversation document and reset on join_room.
- Socket server shares the same Node.js HTTP server as the Express REST API — no separate process.

5. Object-Oriented Programming Concepts

All four OOP principles are demonstrably applied throughout the backend codebase. The following sections document where each principle appears and how it was implemented.

5.1 Encapsulation

Encapsulation bundles data and the methods that operate on it while hiding internal details from the outside world. Three implementations:

Location	What is encapsulated	How
models/User.js	Sensitive fields (password, razorpayAccountId)	getPublicProfile() deletes these before returning data to any caller.
repositories/BaseRepository.js	Mongoose model instance	Stored as private field #model. Subclasses access only via protected getter.
middleware/rateLimiter.js	Per-IP request counter Map	#requests is private. External code only calls .middleware() — never touches state.

5.2 Inheritance

BaseRepository defines all generic database operations. The four domain repositories extend it, inheriting all methods and adding only their own domain-specific queries.

Subclass	Inherits From	Additional Methods Added
UserRepository	BaseRepository	findByEmail, getSavedListings, addToSavedListings, removeFromSavedListings
ListingRepository	BaseRepository	findWithFilters (search/sort/paginate), findBySeller, markAsSold, findByIdWithSeller
OrderRepository	BaseRepository	findByRazorpayOrderId, findBuyerOrders, findSellerOrders, markPaid
NotificationRepository	BaseRepository	findByUser, markAsRead

5.3 Polymorphism

Polymorphism allows objects of different types to be used through a shared interface. Two prominent examples:

5.3.1 Notification Hierarchy — toPayload()

All notification types extend BaseNotification and implement toPayload() differently. NotificationFactory calls toPayload() without knowing which concrete class it holds.

Class	toPayload() produces
OrderNotification	"Your order for [item] of Rs.[amount] has been placed."
PaymentNotification	"Payment of Rs.[amount] received for [item]."
ChatNotification	"[sender] sent you a message."
ListingNotification	"Your listing [item] has been sold!"

5.3.2 Payment Strategy — createOrder / verifyPayment / transferToSeller

RazorpayStrategy and CODStrategy both extend BasePaymentStrategy. PaymentContext calls these three methods through the shared interface, allowing the payment method to change at runtime without any change to PaymentService.

5.4 Abstraction

Abstract base classes define contracts that hide implementation complexity from calling code.

Abstract Class	File	Contract it defines	Who uses it
BaseRepository	repositories/BaseRepository.js	findById, find, create, updateById, deleteById	All 4 service classes via their specific repository
BasePaymentStrategy	patterns/PaymentStrategy.js	createOrder, verifyPayment, transferToSeller	PaymentContext in PaymentService
BaseNotification	patterns/NotificationFactory.js	toPayload()	NotificationFactory and NotificationService

6. Design Patterns Implemented

Four design patterns were implemented with real business justification — none were added artificially. Each pattern is documented with its location, the problem it solved, and the alternative approach that would have been used without it.

6.1 Singleton Pattern

Attribute	Detail
Files	configuration/Database.js | patterns/OrderEventEmitter.js
Category	Creational
Problem solved	Without Singleton, every module that imports the DB connection would create a new connection to MongoDB, exhausting connection limits. Every import of EventEmitter would create a separate bus, so listeners registered in one module would not receive events emitted from another.
Implementation	static #instance = null with a static getInstance() method. The constructor sets Database.#instance = this on first call and returns it on all subsequent calls.
Business value	One stable MongoDB connection for the entire app lifetime. One shared event bus so OrderEventEmitter listeners in PaymentService fire reliably from any module.

6.2 Factory Pattern

Attribute	Detail
File	patterns/NotificationFactory.js
Category	Creational
Problem solved	Callers (PaymentService, NotificationService) need to create notification objects but should not need to know which concrete class to instantiate. Adding a new notification type should not require changes in the calling code.
Implementation	NotificationFactory.create(type, data) switches on type and returns the correct subclass. All subclasses extend BaseNotification and implement toPayload().
Business value	Adding a new notification type (e.g. ReviewNotification) requires only one new class and one new case in the factory. No service files change.

6.3 Observer Pattern

Attribute	Detail
Files	patterns/OrderEventEmitter.js | services/PaymentService.js
Category	Behavioural
Problem solved	After payment verification, multiple subsystems must react: notify the buyer, notify the seller, mark the listing sold, and trigger the Razorpay transfer. Placing all this logic inside verifyAndComplete() creates a bloated, tightly-coupled function.

Attribute	Detail
Implementation	OrderEventEmitter extends Node's built-in EventEmitter. PaymentService calls emitOrderPaid(data) after verification. Two independent listeners registered in the same file call the Factory and save notifications without PaymentService knowing about them.
Business value	Adding a new post-payment action (e.g. send email) requires only a new listener. The verification flow is unchanged.

6.4 Strategy Pattern

Attribute	Detail
File	patterns/PaymentStrategy.js
Category	Behavioural
Problem solved	The payment method must be swappable (Razorpay for live payments, COD for cash) without changing PaymentService. Without Strategy, adding COD would require if-else branches inside verifyAndComplete().
Implementation	BasePaymentStrategy defines createOrder / verifyPayment / transferToSeller. RazorpayStrategy and CODStrategy implement these. PaymentContext holds the active strategy and delegates all calls to it. PaymentService only interacts with PaymentContext.
Business value	Adding a new payment provider (e.g. PhonePe) requires only a new Strategy class. PaymentService and PaymentContext are untouched.

7. SOLID Principles

All five SOLID principles are reflected in the refactored codebase. The table below maps each principle to its exact implementation location.

Principle	Full Name	Implementation in CampusXchange
S	Single Responsibility	Controller (controllers/index.js) handles only HTTP req/res. Service handles only business logic. Repository handles only DB access. Each class has exactly one reason to change. Also: RateLimiter only rate-limits; ErrorHandler only formats errors.
O	Open / Closed	New payment method = new Strategy class only, nothing else changes. New notification type = new class + one case in Factory. BaseRepository is never modified to support new query types — subclasses extend it instead.
L	Liskov Substitution	UserRepository, ListingRepository, OrderRepository, NotificationRepository can each replace BaseRepository wherever a repository is expected without breaking behaviour. RazorpayStrategy and CODStrategy can replace BasePaymentStrategy inside PaymentContext identically.
I	Interface Segregation	Six separate Express routers (authRouter, listingRouter, paymentRouter, orderRouter, notificationRouter, profileRouter). Auth routes never import payment logic. Payment routes never import chat logic. No module depends on methods it does not use.
D	Dependency Inversion	AuthService depends on UserRepository (abstraction), never on mongoose.model(). PaymentService depends on PaymentContext (abstraction), never on the Razorpay SDK directly. Services are instantiated with their dependencies injected, making them independently testable.

8. System Design Optimisations

Optimisation	File / Component	Description
Layered Architecture	Entire backend	Strict 5-layer separation (Route → Controller → Service → Repository → Model). Each layer is independently replaceable and testable.
Rate Limiting	middleware/rateLimiter.js	Custom in-memory RateLimiter class with per-IP tracking. Auth routes: 10 req/15 min. Payment routes: 20 req/min. General: 100 req/min. Prevents brute-force attacks without a third-party library.
Global Error Handling	middleware/errorHandler.js	Centralised AppError class and globalErrorHandler middleware. All errors propagate via next(err). Consistent JSON error response format across every endpoint. Stack trace hidden in production.
Event-Driven Side Effects	patterns/OrderEventEmitter.js	Post-payment actions (notifications, transfers) handled by event listeners. Payment verification is decoupled from notification logic. New post-payment actions require zero changes to PaymentService.
Singleton DB Connection	configuration/Database.js	Guarantees exactly one MongoDB connection for the app lifetime. Prevents duplicate connections on hot reload in development.
Parallel DB Queries	repositories/ListingRepository.js	Promise.all() fetches listing items and total count simultaneously, halving listing query latency vs sequential queries.
JWT Stateless Auth	middleware/authenticate.js	No server-side session storage. Every request carries its own signed token. Horizontally scalable with no shared session state.
Socket Room Isolation	src/index.js	Each conversation is a Socket.IO room. Users join only rooms they are party to (validated against the DB). Prevents message leakage between conversations.
Frontend Service Layer	src/services/api.js	All fetch() calls centralised in 7 service classes. Components never call fetch() directly. Base URL, auth headers, and error formatting handled in one place.
Custom React Hooks	src/hooks/index.js	useListings, useSaved, useProfile, useAuth, useOrders abstract loading/error state from all page components. Pages are pure UI with no data-fetching logic.

9. Features Implemented

9.1 Authentication & User Management

- JWT-based signup and login with bcrypt password hashing.
- Protected routes — all non-public endpoints require a valid Bearer token.
- Profile management: name, phone, hostel, room, bio, profile photo (Cloudinary).
- Profile completion percentage indicator to encourage complete profiles.
- Public user profiles viewable by other students.

9.2 Listing Management

- Create listings with up to 5 images (uploaded to Cloudinary, max 5 MB each, 1000x1000 px limit).
- Full-text search on listing title with case-insensitive regex.
- Filter by category (books, electronics, notes, accessories, others), price range, and condition.
- Sort by newest first, price ascending, or price descending.
- Pagination with configurable page size (default: 20 per page). Total count fetched in parallel.
- Sellers can edit and delete their own listings. Ownership enforced at repository level.
- Listing status transitions: available → sold (automated on successful payment).

9.3 Real-Time Chat

- Buyer initiates a conversation from the product detail page.
- Each conversation is scoped to one listing and one buyer-seller pair.
- Socket.IO rooms ensure messages are delivered only to conversation participants.
- Unread message counters tracked per participant; reset to zero on room join.
- Conversation list shows last message preview and unread count badge.
- Closed conversations (post-sale) are marked and locked.

9.4 Payment System

- Razorpay payment gateway integration — buyer pays via Razorpay popup.
- HMAC SHA-256 signature verification on every payment — prevents fraudulent verification calls.
- Automated Razorpay route transfer to seller's linked account after successful payment.
- Seller Razorpay onboarding flow — generates a one-click account link.
- Payment status tracking: not_enabled → pending_verification → active.
- Platform fee deducted (100 paise) from each transfer.

9.5 Wishlist & Order History

- Toggle save/unsave any listing. Saved listings persisted as ObjectId array on User document.
- Buyer dashboard: list of all purchased items with seller details.
- Seller dashboard: list of all sold items with buyer details.

9.6 Notification System

- Buyer receives notification on successful order placement.
- Seller receives notification when their listing is sold.
- Notifications stored in MongoDB. Fetched per user. Mark-as-read supported.

10. Database Design

10.1 Collections Overview

Collection	Key Fields	Relationships
User	_id, name, email, password, hostel, room, razorpayAccountId, paymentStatus, savedListings[]	1 User sells many Listings. 1 User has many Orders (as buyer and seller). 1 User has many Notifications. Many-to-many with Listing via savedListings.
Listing	_id, title, description, price, category, condition, images[], location, seller (FK), status	Belongs to 1 User (seller). Has many Orders. Discussed in many Conversations.
Order	_id, buyerId (FK), sellerId (FK), listingId (FK), amount, razorpayOrderId, status	References User (x2) and Listing. Created on payment initiation, updated to paid on verification.
Conversation	_id, buyerId (FK), sellerId (FK), listingId (FK), lastMessage, unread{buyer,seller}, closed	References User (x2) and Listing. Contains many Messages.
Message	_id, conversationId (FK), senderId (FK), text, createdAt	Belongs to 1 Conversation. Sent by 1 User.
Notification	_id, user (FK), message, type, link, isRead, createdAt	Belongs to 1 User. Created by Observer listener via Factory.

10.2 Design Decisions

- MongoDB chosen over SQL for its flexible schema — listings have varying fields by category.
- Mongoose ODM used for schema validation, virtuals, and instance methods (e.g. getPublicProfile).
- Mongoose model guard (mongoose.models.X || mongoose.model('X', schema)) prevents OverwriteModelError on hot reload.
- savedListings stored as ObjectId[] on User (not a join collection) — optimised for the read-heavy wishlist page.
- unread counts embedded in Conversation (not counted at query time) — O(1) unread badge reads.

11. UML Diagrams

Four UML diagrams were produced to document the system. Full rendered versions are provided in the accompanying UML Diagrams PDF.

Diagram	What it shows	Key elements
Class Diagram	All classes, their attributes, methods, and relationships.	BaseRepository → 4 repo subclasses (Inheritance). BaseNotification → 4 notification subclasses (Polymorphism). Singleton, Factory, Observer, Strategy pattern classes. 5 Mongoose models with instance methods.
Use Case Diagram	All actors and their interactions with the system.	Actors: Buyer, Seller, System (automated). Use cases: Register, Login, Browse, Purchase, Chat, Manage Listings, Onboard Razorpay, Receive Notifications. «include» from Purchase to Verify Payment to Transfer.
Sequence Diagram	Complete payment workflow across 6 system components.	8 labelled steps from POST /pay to payment confirmation. Shows Strategy (HMAC check), Observer (emit order:paid), Factory (create notifications) in sequence.
ER Diagram	All MongoDB collections, fields, PK/FK, and relationships.	Crow's foot notation. 6 collections. 7 relationships including one many-to-many (wishlist).

12. Test Cases & Results

All test cases were executed manually using Postman for API endpoints and the browser for frontend flows.

#	Test Case	Input	Expected Output	Result
1	User Registration	POST /auth/signup — name, email, password	201 + accessToken	PASS
2	Duplicate Email	POST /auth/signup — existing email	400 User already exists	PASS
3	Login — valid credentials	POST /auth/login — correct email & password	200 + accessToken	PASS
4	Login — wrong password	POST /auth/login — incorrect password	401 Invalid credentials	PASS
5	Create listing	POST /api/listings — auth token + image upload	201 listing object with Cloudinary URL	PASS
6	Listings with filters	GET /api/listings?category=books&sort=price_asc&page=1	200 paginated items + pagination object	PASS
7	Search listings	GET /api/listings?search=calculus	200 matching listings only	PASS
8	Toggle wishlist	POST /api/saved/:listingId — auth token	saved: true / false toggled correctly	PASS
9	Create payment order	POST /api/payments/pay/:listingId — auth token	200 Razorpay order object	PASS
10	Verify payment — valid HMAC	POST /api/payments/verify — correct signature	200 { success: true }	PASS
11	Verify payment — bad HMAC	POST /api/payments/verify — tampered signature	400 Payment verification failed	PASS
12	Auth rate limiter	11 rapid POST /auth/login requests from same IP	429 Too many requests on 11th request	PASS
13	Protected route — no token	GET /api/listings/my — no Authorization header	401 Missing token	PASS
14	Protected route — expired	GET /api/profile — expired token	403 Invalid or expired token	PASS
15	Update own listing	PUT /api/listings/:id — owner's auth token	200 updated listing object	PASS
16	Update other user's listing	PUT /api/listings/:id — different user's token	404 not found or not owner	PASS

#	Test Case	Input	Expected Output	Result
17	Real-time message delivery	Socket emit send_message in conversation room	receive_message emitted to room members	PASS
18	Notification on order	Successful payment verification	Buyer + Seller notifications in DB	PASS
19	Notification Factory types	Observer fires with order:paid	Correct message string per type	PASS
20	Profile update	PUT /api/profile — new hostel/room/bio	200 updated profile without password	PASS

13. Challenges Faced & Solutions

Challenge	Root Cause	Solution Applied
OverwriteModelError on hot reload	Mongoose caches models; re-importing re-registers them.	Added mongoose.models.X || mongoose.model('X', schema) guard on all model exports.
Socket.IO authentication	HTTP middleware does not apply to socket connections.	Added io.use() middleware that reads socket.handshake.auth.token and verifies JWT before connection is accepted.
Razorpay HMAC mismatch in testing	Order ID and payment ID concatenation order was wrong.	Followed Razorpay docs exactly: body = razorpay_order_id + '|' + razorpay_payment_id.
Route conflict — /my vs /:id	Express matched /my as a dynamic :id parameter.	Moved /my route registration above /:id in the router so the static path takes precedence.
Cloudinary upload in multipart requests	JSON body parser conflicts with multipart/form-data.	Used multer-storage-cloudinary which handles multipart parsing independently of express.json().
ES Module import path issues	Copied files retained old import paths (authMiddleware.js vs authenticate.js).	Updated all import paths in chat.js and routes to reference the new middleware locations.

14. Conclusion

CampusXchange successfully demonstrates the practical application of software engineering principles in a real-world, deployable full-stack project. The platform solves a genuine problem faced by university students and delivers a production-ready solution with a working payment gateway, real-time communication, and a complete listing marketplace.

The refactoring exercise transformed a basic MVC application into a production-grade, five-layer architecture. This process made tangible every concept covered in the System Design and OOP course:

- All four OOP principles — Encapsulation, Inheritance, Polymorphism, Abstraction — are applied with real business value, not as academic exercises.
- Four design patterns — Singleton, Factory, Observer, Strategy — solve actual problems in the payment, notification, and database connection domains.
- All five SOLID principles are traceable to specific files and classes in the codebase.
- System Design optimisations — layered architecture, rate limiting, event-driven side effects, stateless auth, parallel queries — improve scalability, security, and maintainability.
- All four UML diagram types document the system comprehensively for both technical and non-technical audiences.
- Twenty test cases verify functional correctness of all major API endpoints and flows.

The project is live at campus-x-change-gamma.vercel.app and the source code is available at github.com/BineetKeshari2005/campusXchange.

End of Report